LAMP: Linear Verification of Matrix Multiplication via Proximity Testing

Anonymous Submission

***Abstract*—Verifiable computation systems often need to prove large matrix multiplication statements, but a direct SNARK arithmetization of a $k \times k$ product requires $\mathcal{O}(k^3)$ constraints. Freivalds' randomized check reduces the algebraic computation to vector-matrix products, but proving those products inside a SNARK still costs $\mathcal{O}(k^2)$ constraints.**

We present LAMP, a matrix-multiplication checking protocol that combines Freivalds' randomized check with proximity testing over linear error-correcting codes. The prover commits to encoded matrices and intermediate vectors before the sampled query positions are derived. The CP-SNARK circuit then checks only the sampled codeword positions and commits to the values used inside the circuit, while Merkle openings and CP-Link proofs ensure consistency between the in-circuit witnesses and the externally committed values. We prove soundness of the public-coin interactive protocol for this committed-input setting under the soundness of the SNARK backend, the binding of the commitments, the correctness of the CP-Link checks, and the distance of the code.

For a fixed number t of sampled positions, the main in-circuit SNARK relation has $\mathcal{O}(tk+E_{\mathsf{code}})$ constraints, which is linear in k for fixed t in our fixed-rate Reed–Solomon instantiation. Merkle openings and CP-Link add $\mathcal{O}(t \log n)+E_{\mathsf{link}}(k,t)$ backend work. We implement LAMP in Go and evaluate the revised protocol against a matched Freivalds-in-Groth16 baseline and the public DualMatrix implementation. The evaluation reports the complete cost of the additional B-proximity test and distinguishes native prover work from in-circuit constraints.

***Index Terms*—Verifiable computation, SNARKs, Proximity testing, Linear codes, Freivalds' algorithm, Merkle trees, Commitment linking**

1. Introduction

Matrix multiplication is a core computational primitive underlying many large-scale computations, including numerical linear algebra, signal processing, computer graphics, and artificial intelligence. Consequently, many applications require verifiability for matrix products while keeping the underlying matrices private. Zero-knowledge proofs provide a cryptographic mechanism for this goal: a prover can convince a verifier that a statement is correct without revealing any secret information that witnesses the statement.

Applying general-purpose ZKP systems directly to large matrix multiplication, however, remains expensive. Succinct proof systems such as [1]–[4] prove relations after the computation has been represented as an arithmetic circuit or a similar algebraic constraint system. A standard arithmetization of a $k \times k$ matrix product $\mathbf{AB} = \mathbf{C}$ computes every output entry and therefore uses $\mathcal{O}(k^3)$ arithmetic constraints. This is already impractical for moderate k, and it is especially problematic for Transformer-based neural networks [5]–[7], whose projection and attention layers contain large matrix multiplications.

The $\mathcal{O}(k^2)$ barrier. Freivalds' classical randomized check [8] reduces matrix-product verification from a cubic computation to a quadratic one. Given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and a random vector $\mathbf{r}$, the verifier checks

$$\mathbf{rAB} = \mathbf{rC}$$

instead of recomputing the full product. This reduces the verification cost from $\mathcal{O}(k^3)$ to $\mathcal{O}(k^2)$. The resulting $\mathcal{O}(k^2)$-size circuit is an improvement, but it remains costly at the dimensions used by modern matrix workloads. Thus, a large gap remains between the quadratic SNARK cost and practical large-scale matrix-multiplication verification.

Our insight: from computation to checking. We observe that the $\mathcal{O}(k^2)$ barrier is not inherent to the verification problem itself. It arises because existing SNARK-based approaches still *compute* the vector–matrix products inside the circuit. Our key idea is to move from "computing inside the circuit" to "*checking* inside the circuit." The prover performs the expensive matrix arithmetic outside the SNARK and commits to the relevant encoded data and intermediate values. The verifier then checks only a small number of lightweight algebraic relations that are sufficient to detect inconsistent claims. The technical core of this approach is proximity testing over linear error-correcting codes. Let $\mathcal{C}$ be a linear code with relative distance δ. We encode each matrix row-wise and consider the Freivalds intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \qquad \mathbf{y} = \mathbf{xB}, \qquad \mathbf{z} = \mathbf{rC}.$$

The linearity of $\mathcal{C}$ lets us check these vector–matrix products in the encoded domain, column by column. For one encoded column, a single inner product, or *fold*, checks one coordinate of the encoded product, and random queries detect inconsistent relations according to the code distance.

The key point is that the SNARK circuit never takes the full matrices as witnesses. Instead, it checks fold equations on t sampled encoded columns, where t is independent of the matrix dimension k, and checks the encoding consistency of the intermediate vectors. To bind the circuit witnesses to the data fixed before the query positions are known, we use a commit-and-prove SNARK for the sampled block witnesses. The prover also supplies Merkle openings and CP-Link proofs that bind the sampled circuit witnesses to the pre-query commitments.

As a result, the main SNARK relation has $\mathcal{O}(tk+E_{\mathsf{code}})$ constraints, where E_{code} is the cost of checking that the intermediate vectors are valid encodings under the error-correcting code. For the fixed-rate Reed–Solomon setting used in our implementation, this remains linear in k for fixed t. Here "linear" refers only to the main in-circuit constraint count: native vector–matrix products and encoded-matrix commitments still require $\mathcal{O}(k^2)$ prover work.

Contributions. We present LAMP, a protocol for efficiently proving matrix-multiplication statements. Our contributions are as follows.

- **A code-based matrix-product checking protocol.** LAMP combines Freivalds' randomized reduction with proximity testing over linear error-correcting codes. Instead of computing vector–matrix products inside the SNARK circuit, the protocol checks sampled fold equations over encoded commitments and verifies code consistency for the intermediate vectors.
  For fixed t, the main circuit cost is $\mathcal{O}(tk + E_{\mathsf{code}})$, which is linear in k for the fixed-rate Reed–Solomon setting used in our implementation.
- **Soundness analysis.** We prove soundness of the public-coin interactive protocol for committed inputs, accounting for input proximity, sampled relation checks, and the commitment-linking backend.
- **Implementation and evaluation.** We implement LAMP in Go and compare the revised construction with a matched Freivalds-in-Groth16 baseline and the public DualMatrix implementation. Our accounting separates native matrix and encoding work, circuit constraints, setup, proving, verification, memory, and proof size. Secondary batch and GPT-2 workloads are reported only when run with the same revised transcript and soundness parameters.

2. Related Work

Matrix multiplication. Several proof systems have studied how to verify matrix multiplication more efficiently than by arithmetizing the full cubic computation. Thaler's sum-check-based protocol [9] shows that matrix multiplication can be verified with quadratic prover work by reducing the claim to checks over low-degree extensions. This avoids the naive $\mathcal{O}(k^3)$ arithmetic cost, but the verifier's work still grows linearly with the matrix dimension, which is costly when the goal is succinct verification for large committed matrices. More recent protocols design proof systems directly for committed matrix multiplication. zkMatrix [10] reduces committed $k \times k$ matrix multiplication to inner-product relations and adapts Bulletproofs-style arguments, achieving $\mathcal{O}(k^2)$ prover work and $\mathcal{O}(\log k)$ proof size and verifier work. Its follow-up, DualMatrix [11], reduces the field-operation cost to $\mathcal{O}(K+k)$, where K is the number of nonzero matrix entries, while using $\mathcal{O}(k)$ group operations. For dense matrices, $K = \Theta(k^2)$. zkMaP [12] instead uses KZG commitments to obtain $\mathcal{O}(k^2)$ prover work, a constant-size proof, and constant verifier work without placing the matrix product in an arithmetic circuit.

Table 1 separates total prover work from in-circuit constraints. This distinction is central to LAMP: computing the intermediate vectors and encoded-column commitments still requires $\mathcal{O}(k^2)$ total prover work, but the matrix-multiplication relation inside the CP-SNARK uses only $\mathcal{O}(tk + E_{\mathsf{code}})$ constraints. For fixed t and $n = \Theta(k)$, its Merkle-dominated proof size and verifier work are $\mathcal{O}(\log k)$. Thus, the table does not claim an unconditional end-to-end advantage over every prior system; it identifies the different cost boundary improved by LAMP.

We found no public zkMatrix implementation, and the zkMaP artifact link was unavailable. We therefore use the public DualMatrix implementation as the closest reproducible dedicated baseline in Section 7. Our Freivalds-in-Groth16 baseline is not state of the art; it uses the same Groth16 stack to isolate the effect of moving vector–matrix products out of the circuit.

Verifiable AI. Matrix multiplication is also a dominant cost in verifiable machine-learning systems. Systems such as vCNN [13], pvCNN [14], and zkVC [15] optimize verifiable inference for convolutional neural networks by reducing the number of multiplication constraints induced by QAP-based representations. Other systems use sum-check-style techniques. zkCNN [16] proposes a sum-check-based protocol for proving convolutions with linear prover time. In addition, zkGPT [17] and zkLLM [18] introduce protocols specialized for LLM architectures and use sum-check protocols to prove the correctness of linear layers efficiently. Meanwhile, Mystique [19] and zkML [20] use Freivalds' randomized checks.

Code-based proximity testing. LAMP also builds on the use of linear error-correcting codes and proximity tests in succinct proof systems. FRI [21] and related works [22], [23] construct proximity IOPs for Reed–Solomon codewords. Recent work further extends this viewpoint to more general foldable codes [24]. Our proximity layer is closer in spirit to the local codeword tests used in Ligero [25], Brakedown [26], and Blaze [27]. In particular, our use of in-circuit encoding checks is similar in spirit to Orion [28]. Follow-up work on Orion's soundness [29] illustrates why codeword-validity and proximity conditions must be discharged explicitly rather than left as input assumptions. Our analysis uses the Reed–Solomon proximity-gap result and the single-parameter batching tradeoff for interleaved tests [30], [31] to state the input-extraction error separately from the sampled relation error. We do not claim novelty for code-based proximity testing itself; our contribution is a matrix-specific reduction that combines these tools with Freivalds' check and commitment linking to reduce in-circuit matrix-product constraints.

3. Preliminaries

Throughout, $\mathbb{F}$ is a prime field and λ is the security parameter. Matrices are denoted by bold uppercase letters and vectors by bold lowercase letters. For $\mathbf{M}$, we write

System	Total prover work	Main circuit constraints	Proof size	Verifier work
zkMatrix [10]	$\mathcal{O}(k^2)$	–	$\mathcal{O}(\log k)$	$\mathcal{O}(\log k)$
DualMatrix [11]	$\mathcal{O}(K+k)\,\mathbb{F}+\mathcal{O}(k)\,\mathbb{G}$	–	$\mathcal{O}(\log k)$	$\mathcal{O}(\log k)$
zkMaP [12]	$\mathcal{O}(k^2)$	–	$\mathcal{O}(1)$	$\mathcal{O}(1)$
LAMP	$\mathcal{O}(k^2)\,\mathbb{F}+\mathcal{O}(k^2)\,\mathbb{G}$	$\mathcal{O}(tk+E_{\mathsf{code}})$	$\mathcal{O}(\log k)$ for fixed t	$\mathcal{O}(\log k)$ for fixed t

TABLE 1. Asymptotic comparison with dedicated committed-matrix-multiplication proofs. A dash means that the approach does not use a separate arithmetic circuit for the matrix-multiplication relation. For DualMatrix, $K=\Theta(k^2)$ on dense matrices.

$\mathbf{M}[i,*]$ and $\mathbf{M}[*,j]$ for its i-th row and j-th column. We use $[n]=\{0,\dots,n-1\}$, $I \leftarrow\!\!\$\ [n]^t$ for t uniform query positions, and $r \leftarrow\!\!\$\ \mathbb{F}$ for a uniform field element.

3.1. Linear Codes and Folding

Let $\mathcal{C}:\mathbb{F}^k\to\mathbb{F}^n$ be a linear code with generator matrix $\mathbf{G}$, rate $\rho=k/n$, and relative distance δ. We write $\mathsf{Enc}(\mathbf{m})=\mathbf{m}\mathbf{G}$. A matrix $\mathbf{M}\in\mathbb{F}^{k\times k}$ is encoded row-wise as $\widehat{\mathbf{M}}=\mathsf{Enc}(\mathbf{M})\in\mathbb{F}^{k\times n}$.

Definition 3.1 (Interleaved code)**.** The ℓ-fold interleaved code $\mathcal{C}^{\langle\ell\rangle}$ groups ℓ codewords position by position. Its j-th symbol is an element of $\mathbb{F}^{\ell}$. In particular, a row-wise encoded $k\times k$ matrix is a k-fold interleaved codeword whose j-th symbol is $\widehat{\mathbf{M}}[*,j]$.

For coefficients $r_0,\dots,r_{k-1}\in\mathbb{F}$, code linearity gives

$$\sum_{i=0}^{k-1} r_i\widehat{\mathbf{M}}[i,*]=\mathsf{Enc}\left(\sum_{i=0}^{k-1} r_i\mathbf{M}[i,*]\right). \tag{1}$$

For $\mathbf{r},\mathbf{c}\in\mathbb{F}^k$, define

$$\mathsf{Fold}(\mathbf{r},\mathbf{c}) \coloneqq \langle\mathbf{r},\mathbf{c}\rangle. \tag{2}$$

Thus $\mathsf{Fold}(\mathbf{r},\widehat{\mathbf{M}}[*,j])$ is the j-th coordinate of $\mathsf{Enc}(\mathbf{r}\mathbf{M})$. Our implementation uses a Reed–Solomon code; Appendix B gives its concrete consistency check.

3.2. Commitment and Proof Interfaces

We use Merkle trees as position-binding vector commitments and Pedersen commitments as hiding, computationally binding, and additively homomorphic commitments. We write $\mathsf{MT.Com}$, $\mathsf{MT.Open}$, and $\mathsf{MT.Verify}$ for Merkle commitment, opening, and verification, and $\mathsf{Ped.Com}$ for Pedersen commitment. Merkle leaves in our construction are Pedersen commitments, so the tree itself uses deterministic leaves.

A commit-and-prove SNARK (CP-SNARK) [32], [33] proves an NP relation while binding a designated witness part u to a public witness commitment $\widetilde{\mathsf{cm}}=\mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Com}(\mathsf{pp},\mathsf{u};\widetilde{o})$. We use the interface $\mathbf{\Pi}_{\mathsf{cp}}=(\mathsf{Setup},\mathsf{Com},\mathsf{Prove},\mathsf{Verify})$.

CP-Link proves that a SNARK-side commitment to an ordered concatenation of sampled blocks contains the same messages as the corresponding external Pedersen commitments. For blocks $(\mathbf{m}_i)_{i=0}^{q-1}$, it links

$$\widetilde{\mathsf{cm}}=\mathsf{Com}(\mathsf{ck}_S,\mathbf{m}_0\|\cdots\|\mathbf{m}_{q-1};\widetilde{o}),$$
$$\mathsf{cm}_i=\mathsf{Com}(\mathsf{ck}_E,\mathbf{m}_i;o_i)\quad (i\in[q]).$$

The construction requires CP-SNARK soundness and zero knowledge, commitment binding and hiding, and CP-Link soundness and witness zero knowledge; these assumptions are summarized in Definition 6.1.

3.3. Structured Freivalds Check

Freivalds' algorithm [8] verifies $\mathbf{AB}=\mathbf{C}$ by checking $\mathbf{rAB}=\mathbf{rC}$. We use the compact structured challenge $\mathbf{r}=(1,r,\dots,r^{k-1})$ for $r\leftarrow\!\!\$\ \mathbb{F}$. If $\mathbf{D}=\mathbf{AB}-\mathbf{C}\neq 0$, a nonzero column of $\mathbf{D}$ defines a nonzero univariate polynomial in r of degree at most $k-1$. By Schwartz–Zippel [34], [35],

$$\Pr[\mathbf{rD}=0]\le\frac{k-1}{|\mathbb{F}|}.$$

4. Overview of LAMP

This section gives a technical overview of LAMP before the formal construction in Section 5. The main message is that LAMP does not ask the SNARK circuit to compute a matrix product. Instead, the prover commits to encoded data, and the circuit checks only a small number of local consistency equations at randomly sampled codeword positions.

4.1. Problem Statement and Relation Reduction Chain

The starting point is the matrix multiplication relation

$$\mathbf{AB}=\mathbf{C},\qquad \mathbf{A},\mathbf{B},\mathbf{C}\in\mathbb{F}^{k\times k}.$$

A direct SNARK arithmetization of this relation computes every entry of $\mathbf{AB}$. This requires k^3 scalar multiplications and therefore $\mathcal{O}(k^3)$ arithmetic constraints. This is the original relation:

$$\mathcal{R}_{\mathsf{orig}}=\{(\mathbf{A},\mathbf{B},\mathbf{C}):\mathbf{AB}=\mathbf{C}\}.$$

Freivalds' algorithm reduces the cubic check to a vector-matrix check. The classical algorithm samples a challenge vector $\mathbf{r}\in\mathbb{F}^k$ and checks $\mathbf{rAB}=\mathbf{rC}$.

In LAMP, the verifier derives one scalar r and uses the structured vector $(1,r,\dots,r^{k-1})$, which keeps the public challenge compact. At the same transcript point, it derives an independent scalar s and the structured vector $\mathbf{s} = (1, s, \ldots, s^{k-1})$. This second challenge is used only to test the committed encoding of $\mathbf{B}$. Equivalently, the prover introduces intermediate vectors

$$\mathbf{x} = \mathbf{r}\mathbf{A}, \qquad \mathbf{y} = \mathbf{x}\mathbf{B}, \qquad \mathbf{z} = \mathbf{r}\mathbf{C},$$

and the verifier checks $\mathbf{y} = \mathbf{z}$. This gives the Freivalds relation

$$\mathcal{R}_{\mathsf{Fr}} = \Big\{(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{x}, \mathbf{y}, \mathbf{z}) : \\ \mathbf{x} = \mathbf{r}\mathbf{A}, \quad \mathbf{y} = \mathbf{x}\mathbf{B}, \quad \mathbf{z} = \mathbf{r}\mathbf{C}, \quad \mathbf{y} = \mathbf{z}\Big\}.$$

This removes the cubic matrix-matrix computation, but a SNARK circuit that computes $\mathbf{r}\mathbf{A}$, $\mathbf{x}\mathbf{B}$, and $\mathbf{r}\mathbf{C}$ directly still performs $\mathcal{O}(k^2)$ scalar multiplications.

LAMP applies one more reduction. Rather than computing the full vector-matrix products inside the circuit, it checks the products only at randomly sampled positions in an encoded domain. The prover performs the expensive native computation outside the SNARK, commits to the encoded matrices and intermediate vectors, and the SNARK checks local equations of the form

$$\mathsf{Enc}(\mathbf{x})[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*, j]),$$
$$\mathsf{Enc}(\mathbf{y})[j] = \mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*, j]),$$
$$\mathsf{Enc}(\mathbf{z})[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*, j]).$$

The prover additionally computes $\mathbf{b} = \mathbf{s}\mathbf{B}$, encodes it as $\widehat{\mathbf{b}} = \mathsf{Enc}(\mathbf{b})$, and the circuit checks

$$\widehat{\mathbf{b}}[j] = \mathsf{Fold}(\mathbf{s}, \widehat{\mathbf{B}}[*, j]).$$

This additional fold tests the committed encoding of $\mathbf{B}$ independently of the product relation. Only the queried columns $\widehat{\mathbf{A}}[*, j]$, $\widehat{\mathbf{B}}[*, j]$, $\widehat{\mathbf{C}}[*, j]$ are used by the circuit. Thus, for t queried positions, the fold part of the circuit has cost $\mathcal{O}(tk)$, which is linear in k for fixed t.

4.2. Encoding and Proximity Testing

Let $\mathcal{C} : \mathbb{F}^k \to \mathbb{F}^n$ be a linear code with relative distance δ. For a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$, the prover row-wise encodes $\mathbf{M}$ as $\widehat{\mathbf{M}} = \mathsf{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$. By Definition 3.1, this row-wise encoding can also be viewed as a k-fold interleaved codeword, where the j-th interleaved symbol is the encoded column $\widehat{\mathbf{M}}[*, j]$.

The key identity is the linearity of the code. For any vector $\mathbf{u} \in \mathbb{F}^k$,

$$\big(\mathsf{Fold}(\mathbf{u}, \widehat{\mathbf{M}}[*, 1]), \ldots, \mathsf{Fold}(\mathbf{u}, \widehat{\mathbf{M}}[*, n])\big) = \mathsf{Enc}(\mathbf{u}\mathbf{M}).$$

Therefore, one coordinate of the encoded vector-matrix product can be checked using one encoded column. For example, if $\mathbf{x} = \mathbf{r}\mathbf{A}$, then for every $j \in [n]$,

$$\widehat{\mathbf{x}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*, j]).$$

Similarly, if $\mathbf{y} = \mathbf{x}\mathbf{B}$ and $\mathbf{z} = \mathbf{r}\mathbf{C}$, then

$$\widehat{\mathbf{y}}[j] = \mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*, j]), \qquad \widehat{\mathbf{z}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*, j]).$$

The prover commits to the encoded input matrices and, after (r, s), to the complete intermediate messages and encoded views before the query positions are sampled. After the query set $I = (j_1, \ldots, j_t)$ is derived, the SNARK checks the above equations only for $j \in I$, together with the equality

$$\widehat{\mathbf{y}}[j] = \widehat{\mathbf{z}}[j].$$

The SNARK also checks that $\widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}$ are valid encodings of the intermediate vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{b}$. These code-consistency checks are separate from the sampled fold checks and prevent the prover from using arbitrary length-n words as the intermediate encoded views.

The sampled equations have two related roles. The r-folds for $\mathbf{A}, \mathbf{C}$ and the independent s-fold for $\mathbf{B}$ test that the committed input words are close to row-wise encodings. Once the inputs are uniquely decoded, a false vector relation induces two distinct codewords, which disagree on at least a δ-fraction of positions. Section 6 states the corresponding proximity and sampling errors.

4.3. Sampled-Opening Layer

The sampled checks above are meaningful only if the values used inside the SNARK are bound to data fixed before the relevant challenges are sampled. LAMP achieves this through a sampled-opening layer. The prover first commits to the interleaved symbols, equivalently the encoded matrix columns, and fixes a Merkle root before the Freivalds and independent input-proximity challenges are derived. After the challenges are fixed, the prover computes the intermediate vectors and makes one binding commitment to the messages and their complete encoded views before the sampled positions and code-check points are derived.

The SNARK checks only the sampled field equations and intermediate-vector encoding checks. Merkle openings authenticate the sampled input-column leaves. CP-Link binds those leaves to the sampled CP-SNARK block and separately binds the complete intermediate CP-SNARK witness to its pre-query commitment. Consequently, the expensive authentication and linking checks remain outside the circuit. Section 6 gives the formal soundness argument.

5. The LAMP Protocol

Section 4 reduced the original matrix-multiplication relation to the Freivalds relation and then to sampled encoded checks. This section formalizes the relation checked by LAMP. The important point is that LAMP is not a single arithmetic-circuit relation: the arithmetic checks are proved by an inner CP-SNARK, while Merkle membership and CP-Link are verified outside the circuit.

5.1. The Composite LAMP Relation

Let $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$, and let $\mathcal{C} : \mathbb{F}^k \to \mathbb{F}^n$ be a linear code with relative distance δ. The target matrix-multiplication relation is

$$\mathcal{R}_{\mathsf{orig}} = \{(\mathbf{A}, \mathbf{B}, \mathbf{C}) : \mathbf{AB} = \mathbf{C}\}.$$

The verifier does not receive the original matrices as public input. Instead, the prover commits to row-wise encodings of the matrices and proves that the committed objects satisfy the multiplication relation.

The relation checked by LAMP is the composite relation

$$\mathcal{R}_{\mathsf{LAMP}} = \mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}} \wedge \mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}.$$

Here $\mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}}$ is the online relation proved by the CP-SNARK after the Freivalds challenge, query positions, and code-check points are sampled. The relation $\mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}$ is the offline relation: it captures the values fixed by commitments before the online queries are known, and the verifier checks it through Merkle openings and CP-Link proofs. Sections 5.3 and 5.4 define these two relations explicitly. The circuit checks field equations on sampled encoded values, while the auxiliary verifier checks that those values are the ones bound by the pre-query commitments.

Using the original matrices in an application circuit. The standard construction avoids placing all matrix entries in the CP-SNARK witness: the external root binds the encoded matrices, and the CP-SNARK commits to the $\mathcal{O}(tk)$ sampled input columns and the $\mathcal{O}(k + n)$-size intermediate block. If an application circuit already uses $\mathbf{A}, \mathbf{B}, \mathbf{C}$, a commit-and-prove backend can instead commit to all $\mathcal{O}(k^2)$ entries and reuse the same witness variables in the application and LAMP relations. With a systematic encoding, the application reads these entries from designated codeword coordinates; a non-systematic encoding must additionally enforce and account for the encoding-to-message map. If the backend natively binds queried encoded coordinates to this full pre-query witness commitment, the separate Merkle/CP-Link bridge to an independently committed application witness is unnecessary. The witness-commitment work is quadratic, while the matrix-multiplication checks remain $\mathcal{O}(tk + E_{\mathsf{code}})$ constraints. Binding the commitment to the intended application data remains application-specific.

5.2. Commitment Layout and Public Challenges

The prover first encodes the matrices row-wise: $\widehat{\mathbf{A}} = \mathsf{Enc}(\mathbf{A}), \widehat{\mathbf{B}} = \mathsf{Enc}(\mathbf{B}), \widehat{\mathbf{C}} = \mathsf{Enc}(\mathbf{C})$. For each codeword position $j \in [n]$, it packs the three k-dimensional interleaved symbols, equivalently the three encoded columns, into $\mathbf{m}_{ABC,j} := (\widehat{\mathbf{A}}[*, j] \,\|\, \widehat{\mathbf{B}}[*, j] \,\|\, \widehat{\mathbf{C}}[*, j]) \in \mathbb{F}^{3k}$. Using a Pedersen commitment key ck_{ABC}, the prover samples $o_{ABC,j} \leftarrow\!\!\$\ \mathbb{F}$ and computes $\mathsf{cm}_{ABC,j} \leftarrow \mathsf{Ped.Com}(\mathsf{ck}_{ABC}, \mathbf{m}_{ABC,j}; o_{ABC,j})$. The matrix root is the Merkle commitment to these Pedersen commitments: $\mathsf{rt}_{ABC} \leftarrow \mathsf{MT.Com}((\mathsf{cm}_{ABC,j})_{j\in[n]}; \perp)$. Here and below, $\perp$ indicates that no additional randomness is used in the Merkle tree, since the leaves are Pedersen commitments that already provide hiding. The root rt_{ABC} is fixed before the challenge pair (r, s) is derived.

After receiving rt_{ABC}, the verifier independently samples $r, s \leftarrow\!\!\$\ \mathbb{F}$. The prover sets $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$ and $\mathbf{s} = (1, s, s^2, \ldots, s^{k-1})$ and computes $\mathbf{x} = \mathbf{rA}, \mathbf{y} = \mathbf{xB}, \mathbf{z} = \mathbf{rC}, \mathbf{b} = \mathbf{sB}$. It then encodes the intermediate vectors: $\widehat{\mathbf{x}} = \mathsf{Enc}(\mathbf{x}), \widehat{\mathbf{y}} = \mathsf{Enc}(\mathbf{y}), \widehat{\mathbf{z}} = \mathsf{Enc}(\mathbf{z}), \widehat{\mathbf{b}} = \mathsf{Enc}(\mathbf{b})$. The prover next packs the four messages and their encoded views into one intermediate block

$$\mathbf{m}_{XYZB} := \left(\mathbf{x}\|\mathbf{y}\|\mathbf{z}\|\mathbf{b}\|\widehat{\mathbf{x}}\|\widehat{\mathbf{y}}\|\widehat{\mathbf{z}}\|\widehat{\mathbf{b}}\right) \in \mathbb{F}^{4(k+n)}.$$

Using a Pedersen commitment key ck_{XYZB}, it samples $o_{XYZB} \leftarrow\!\!\$\ \mathbb{F}$, computes

$$\mathsf{cm}_{XYZB} \leftarrow \mathsf{Ped.Com}(\mathsf{ck}_{XYZB}, \mathbf{m}_{XYZB}; o_{XYZB}),$$

and sends cm_{XYZB} to the verifier. This commitment fixes both the intermediate messages and the complete words checked by $\mathsf{EncCheck}_{\mathcal{C}}$ before its random points are sampled.

The fourth vector supplies an independent fold of the committed encoding of $\mathbf{B}$; its soundness role is analyzed in Section 6.

Only after receiving cm_{XYZB} does the verifier sample the query tuple $I = (j_1, \ldots, j_t) \leftarrow\!\!\$\ [n]^t$. It also samples public code-check points $\alpha_x, \alpha_y, \alpha_z, \alpha_b \leftarrow\!\!\$\ \mathbb{F} \setminus D$, where D is the code evaluation domain. These points are used by the circuit to check that $\widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}$ are encodings of the claimed intermediate vectors. For the Reed–Solomon code used in our implementation, this check is the barycentric out-of-domain consistency test recalled in Appendix B.

5.3. The Online Relation

The online relation is the arithmetic relation proved inside the inner CP-SNARK. Its role is not to verify Merkle paths or Pedersen openings. Instead, it checks only the field equations that certify the sampled encoded positions are consistent with the Freivalds reduction.

The public statement contains the input root, the intermediate commitment, and verifier-sampled challenges:

$$\mathsf{x} = \left(\mathsf{rt}_{ABC}, \mathsf{cm}_{XYZB}, r, s, I, \alpha_x, \alpha_y, \alpha_z, \alpha_b\right).$$

These commitments bind the CP-SNARK proof to the same transcript and offline checks; the online field equations themselves do not evaluate Pedersen commitments or Merkle paths.

The complete SNARK witness is $\mathsf{w} = (\mathsf{u}, \omega)$, where the committed part of the witness is

$$\mathsf{u} = \left((\mathbf{m}_{ABC,j})_{j\in I}, \mathbf{m}_{XYZB}\right),$$

and ω contains any backend-specific opening randomness. The CP-SNARK treats u as the committed part of the witness. The prover first computes the SNARK-side witness commitments $\widetilde{\mathsf{cm}}_{ABC}$ and $\widetilde{\mathsf{cm}}_{XYZB}$ using $\mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Com}$, and then proves the online relation with witness $\mathsf{w} = (\mathsf{u}, \omega)$. These witness commitments are not checked against the external commitments inside the circuit. Instead, the offline relation connects the sampled ABC block to the corresponding

$$
\begin{array}{l}
\circ\ \mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}}(\mathsf{x};\mathsf{w}):\\
\quad \text{parse}\\
\qquad \mathsf{x} = \big(\mathsf{rt}_{ABC}, \mathsf{cm}_{XYZB}, r, s, I, \alpha_x, \alpha_y, \alpha_z, \alpha_b\big)\\
\qquad \mathsf{w} = (\mathsf{u}, \omega)\\
\qquad \mathsf{u} = \big((\mathbf{m}_{ABC,j})_{j\in I}, \mathbf{m}_{XYZB}\big)\\
\qquad \text{parse } \mathbf{m}_{XYZB} \text{ as } \big(\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{b}, \widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}\big)\\
\quad \mathbf{r} \leftarrow (1, r, \ldots, r^{k-1})\\
\quad \mathbf{s} \leftarrow (1, s, \ldots, s^{k-1})\\
\quad \mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{x}}, \mathbf{x}; \alpha_x) = 1\\
\quad \mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{y}}, \mathbf{y}; \alpha_y) = 1\\
\quad \mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{z}}, \mathbf{z}; \alpha_z) = 1\\
\quad \mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{b}}, \mathbf{b}; \alpha_b) = 1\\
\quad \forall j \in I:\\
\qquad \text{parse } \mathbf{m}_{ABC,j} \text{ as } \big(\widehat{\mathbf{A}}[*,j] \,\|\, \widehat{\mathbf{B}}[*,j] \,\|\, \widehat{\mathbf{C}}[*,j]\big)\\
\qquad \widehat{\mathbf{x}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*,j])\\
\qquad \widehat{\mathbf{y}}[j] = \mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*,j])\\
\qquad \widehat{\mathbf{z}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*,j])\\
\qquad \widehat{\mathbf{b}}[j] = \mathsf{Fold}(\mathbf{s}, \widehat{\mathbf{B}}[*,j])\\
\qquad \widehat{\mathbf{y}}[j] = \widehat{\mathbf{z}}[j]
\end{array}
$$

Figure 1. The Online Relation

Merkle leaves and connects the complete intermediate block to cm_{XYZB}, as specified in Section 5.4. For the CP-Link statements below, let $\mathsf{ck}_{\mathsf{cp},T}$ denote the public commitment key used by the CP-SNARK witness commitment $\widetilde{\mathsf{cm}}_T$, for $T \in \{ABC, XYZB\}$. Write $\mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{v}}, \mathbf{v}; \alpha) = 1$ when the out-of-domain code-consistency check accepts that $\widehat{\mathbf{v}}$ is consistent with $\mathsf{Enc}(\mathbf{v})$ at the public point α. The online relation is defined in Figure 1.

The encoding checks validate the four pre-query-committed intermediate views, and the remaining equations check the sampled folds and $\widehat{\mathbf{y}}[j] = \widehat{\mathbf{z}}[j]$. Binding the sampled input columns and the complete intermediate block to their pre-query commitments is the role of the offline relation.

5.4. The Offline Relation

The sampled input-column link statement is

$$\mathsf{x}^{ABC}_{\mathsf{link}} := \big((\mathsf{ck}_{\mathsf{cp},ABC}, \widetilde{\mathsf{cm}}_{ABC}), (\mathsf{ck}_{ABC}, \mathsf{cm}_{ABC,j})_{j\in I}\big),$$

whereas the full intermediate-block link statement is

$$\begin{aligned}\mathsf{x}^{XYZB}_{\mathsf{link}} := \big(&(\mathsf{ck}_{\mathsf{cp},XYZB}, \widetilde{\mathsf{cm}}_{XYZB}),\\ &(\mathsf{ck}_{XYZB}, \mathsf{cm}_{XYZB})\big).\end{aligned}$$

The offline relation connects the witnesses used by the CP-SNARK to the commitments fixed before the query positions were sampled. These linking and opening checks are verified outside the SNARK circuit.

$$
\begin{array}{l}
\circ\ \mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}\big(\ \mathsf{rt}_{ABC}, \mathsf{cm}_{XYZB}, I, \widetilde{\mathsf{cm}}_{ABC}, \widetilde{\mathsf{cm}}_{XYZB},\\
\quad \pi^{ABC}_{\mathsf{link}}, \pi^{XYZB}_{\mathsf{link}},\\
\quad (\mathsf{cm}_{ABC,j}, \pi^{ABC}_{\mathsf{mem},j})_{j\in I}\big):\\
\quad \forall j \in I:\\
\qquad \mathsf{MT.Verify}(\mathsf{rt}_{ABC}, j, \mathsf{cm}_{ABC,j}, \pi^{ABC}_{\mathsf{mem},j}) = 1\\
\\
\quad \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}^{ABC}_{\mathsf{link}}, \pi^{ABC}_{\mathsf{link}}) = 1\\
\quad \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}^{XYZB}_{\mathsf{link}}, \pi^{XYZB}_{\mathsf{link}}) = 1
\end{array}
$$

Figure 2. The Offline Relation

For each queried position $j \in I$, the verifier receives a Merkle opening for the external Pedersen leaf $\mathsf{cm}_{ABC,j}$ under rt_{ABC}. CP-Link then proves that $\widetilde{\mathsf{cm}}_{ABC}$ contains the ordered concatenation $\mathbf{m}_{ABC,j_1}\|\cdots\|\mathbf{m}_{ABC,j_t}$ from those leaves. A second CP-Link proof connects $\widetilde{\mathsf{cm}}_{XYZB}$ to the single pre-query commitment cm_{XYZB}, thereby binding every message and every symbol used by the four encoding checks. Appendix C gives the corresponding blockwise relation.

The offline relation is defined in Figure 2.

Together, these checks establish $\widetilde{\mathsf{cm}}_{ABC} \leftrightarrow (\mathsf{cm}_{ABC,j})_{j\in I} \leftrightarrow \mathsf{rt}_{ABC}$ and $\widetilde{\mathsf{cm}}_{XYZB} \leftrightarrow \mathsf{cm}_{XYZB}$. Hence the CP-SNARK uses sampled input columns and complete intermediate words that were fixed before I and the code-check points, without verifying Pedersen openings or Merkle paths inside the circuit.

5.5. Protocol Algorithms and Complexity

The public-coin transcript follows $\mathsf{rt}_{ABC} \to (r, s) \to \mathsf{cm}_{XYZB} \to (I, \alpha_x, \alpha_y, \alpha_z, \alpha_b)$. The implementation applies the Fiat–Shamir transform in this order using domain-separated hashes.

The proof consists of the CP-SNARK proof, sampled ABC leaf commitments and Merkle openings, and two CP-Link proofs. The sampled matrix-column blocks and the full intermediate block are private committed witnesses; they are not serialized as public proof material. The public statement contains rt_{ABC}, cm_{XYZB}, and the verifier-sampled challenges.

The ABC CP-Link statement contains the SNARK-side witness commitment and the t corresponding external leaves. The XYZB statement links the complete SNARK-side intermediate witness to its single pre-query commitment.

Cost model. We report the cost of LAMP. The computation of the claimed output matrix $C = AB$ is outside this cost model. We do count the work performed by the LAMP prover after the matrices are available, including encoding, commitment generation, computation of the Freivalds intermediate witnesses, sampled Merkle openings, CP-SNARK proving, and CP-Link proving.

Let $E_{\mathsf{Enc}}(k,n)$ denote the cost of encoding one length-k vector into one length-n codeword. Let $E_{\mathsf{code}} = E_{\mathsf{code}}(k,n)$ denote the constraint cost of the intermediate code-consistency checks; for the Reed–Solomon check in Appendix B, this is $\mathcal{O}(k+n)$ after domain-dependent coefficients are precomputed. We write $\mathbb{G}$ for group operations used by Pedersen commitments, $\mathbb{F}$ for field operations, and $\mathbb{H}$ for hash evaluations. Let $E^{\mathcal{P}}_{\mathsf{snark}}$ and $E^{\mathcal{V}}_{\mathsf{snark}}$ denote the CP-SNARK proving and verification costs, respectively. Let $E^{\mathcal{P}}_{\mathsf{link}}(k,t)$ and $E^{\mathcal{V}}_{\mathsf{link}}(k,t)$ denote the prover and verifier costs, respectively, of the selected CP-Link construction. The concrete QALink costs depend on the linked block lengths and on whether the two statements are batched; Section 7 reports the instantiated costs rather than treating the previous sampled-only layout as unchanged.

Table 2 summarizes the asymptotic costs at the level of the main protocol components.

Component	Prover	Verifier
ABC encoding	$3kE_{\mathsf{Enc}}(k,n)$	–
ABC commit	$\mathcal{O}(3nk)\,\mathbb{G} + \mathcal{O}(n)\,\mathbb{H}$	–
Intermediate	$\mathcal{O}(k^2)\,\mathbb{F}$	–
XYZB encoding	$4E_{\mathsf{Enc}}(k,n)$	–
XYZB commit	$\mathcal{O}(k+n)\,\mathbb{G}$	–
Merkle openings	$\mathcal{O}(t\log n)\mathbb{H}$	$\mathcal{O}(t\log n)\mathbb{H}$
CP-SNARK	$\mathcal{O}(tk) + E_{\mathsf{code}}$ constraints $E^{\mathcal{P}}_{\mathsf{snark}}$	$E^{\mathcal{V}}_{\mathsf{snark}}$
CP-Link	$E^{\mathcal{P}}_{\mathsf{link}}(k,t)$	$E^{\mathcal{V}}_{\mathsf{link}}(k,t)$

TABLE 2. ASYMPTOTIC COSTS FOR LAMP.

Prover work. The prover first encodes the three input matrices and commits to the packed encoded matrix columns. The ABC commitment phase consists of n Pedersen commitments to blocks of length $3k$, followed by a Merkle root over the resulting leaves. After the independent challenges (r,s) are sampled, the prover computes the intermediate witnesses $\mathbf{x} = \mathbf{r}\mathbf{A}, \mathbf{y} = \mathbf{x}\mathbf{B}, \mathbf{z} = \mathbf{r}\mathbf{C}, \mathbf{b} = \mathbf{s}\mathbf{B}$, which costs $\mathcal{O}(k^2)$ field operations. This is distinct from the excluded native computation of $C = AB$. The prover then encodes $\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{b}$, and commits to these messages and their complete encoded views in cm_{XYZB}. It then opens only the sampled ABC Merkle paths, proves the online CP-SNARK relation, and produces the two CP-Link proofs.

For each queried position $j \in I$, the CP-SNARK circuit checks four length-k fold equations:

$$\mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*,j]), \quad \mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*,j]),$$
$$\mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*,j]), \quad \mathsf{Fold}(\mathbf{s}, \widehat{\mathbf{B}}[*,j]).$$

Thus the sampled arithmetic checks contribute $\mathcal{O}(tk)$ constraints.

The intermediate-vector code-consistency checks add E_{code} constraints. For the fixed-rate Reed–Solomon setting used in the experiments, $n = 2k$, so this extra cost is linear in k and the total online relation remains linear in k for fixed t.

Verifier work. The verifier does not encode matrices, compute intermediate vectors, or build commitment roots. Its work consists of CP-SNARK verification, verification of the sampled Merkle authentication paths, and CP-Link verification. Therefore, using the notation above, the verifier cost is $E^{\mathcal{V}}_{\mathsf{snark}} + \mathcal{O}(t\log n)\mathbb{H} + E^{\mathcal{V}}_{\mathsf{link}}(k,t)$.

Comparison. A naive SNARK arithmetization of matrix multiplication costs $\mathcal{O}(k^3)$ constraints, while a Freivalds-based SNARK baseline computes the vector-matrix products inside the circuit and costs $\mathcal{O}(k^2)$ constraints. LAMP moves the vector-matrix products to native prover work and checks sampled encoded-column relations plus intermediate code-consistency equations inside the CP-SNARK. As a result, the main circuit constraint count is $\mathcal{O}(tk + E_{\mathsf{code}})$, which is linear in k for the fixed-rate code used in our implementation.

6. Security Analysis

We analyze the public-coin interactive protocol. The root rt_{ABC} fixes three arbitrary interleaved words before the independent scalar challenges (r,s) are sampled. The commitment cm_{XYZB} then fixes the four intermediate messages and their complete encoded views before the query set I and the code-check points are sampled.

Our input test is the standard interleaved-to-word reduction used in code-based protocols such as Ligero and Brakedown [25], [26]. Because LAMP uses the powers of one random scalar rather than a fully uniform coefficient vector, we use the one-parameter Reed–Solomon proximity gap of Ben-Sasson et al. [30, Theorem 1.5]; the randomness–soundness tradeoff of this form of batching is also discussed by Diamond and Posen [31]. For a code $\mathcal{C}$ of relative distance δ, a word $W \in \mathbb{F}^{k\times n}$, and $\mathbf{a} = (1, a, \ldots, a^{k-1})$, write

$$\mathsf{FoldWord}(\mathbf{a}, W)[j] := \mathsf{Fold}(\mathbf{a}, W[*,j]).$$

For $0 < \tau < \delta/2$, Theorems 1.2 and 1.5 of Ben-Sasson et al. [30] imply that if W is farther than τ from the k-fold interleaved code, then

$$\Pr_{a\leftarrow\$\mathbb{F}}[\Delta(\mathsf{FoldWord}(\mathbf{a}, W), \mathcal{C}) \le \tau] \le \frac{(k-1)n}{|\mathbb{F}|}.$$

Definition 6.1 (Backend assumptions)**.** The CP-SNARK, sampled openings, and CP-Link are sound with errors ϵ_{cp}, ϵ_{open}, and ϵ_{link}, respectively. In particular, CP-Link binds the complete intermediate witness used by the four encoding checks to cm_{XYZB}, and binds the sampled input columns to the leaves under rt_{ABC}. The four intermediate-vector encoding checks have error ϵ_{code}; for the Reed–Solomon check of Appendix B,

$$\epsilon_{\mathsf{code}} \le \frac{4(n-1)}{|\mathbb{F}| - n}.$$

For zero knowledge, the CP-SNARK and CP-Link are zero knowledge and the Pedersen commitments are hiding.

Theorem 6.2 (Completeness, soundness, and zero knowledge)**.** *Let $\mathcal{C}$ have relative distance δ, choose $0 < \tau <$*

$\delta/2$ in the unique-decoding regime of the cited Reed–Solomon proximity-gap theorem, and assume Definition 6.1. If rt_{ABC} is fixed before (r, s) and cm_{XYZB} is fixed before $(I, \alpha_x, \alpha_y, \alpha_z, \alpha_b)$, then the interactive LAMP *protocol has perfect completeness and zero knowledge. Moreover, for any PPT prover, let* BadInput *denote the event that an input word is farther than τ from the interleaved code or that the uniquely decoded matrices satisfy $\mathbf{AB} \neq \mathbf{C}$. Then*

$$\Pr[\mathsf{Acc} \wedge \mathsf{BadInput}] \leq \epsilon_{\mathsf{cp}} + \epsilon_{\mathsf{open}} + \epsilon_{\mathsf{link}} + \epsilon_{\mathsf{code}} + \frac{3(k-1)n}{|\mathbb{F}|} + 3(1-\tau)^t + \frac{k-1}{|\mathbb{F}|} + 4(1-\delta+\tau)^t.$$

Proof sketch. Condition on the soundness of the CP-SNARK, openings, CP-Link, and encoding checks. The sampled input columns and the complete intermediate words used by the circuit are then the values fixed before the query challenges.

The r-folds test the committed words for $\mathbf{A}$ and $\mathbf{C}$. The vector $\mathbf{x} = \mathbf{rA}$ need not be uniform, so its product equation alone does not test the committed word for $\mathbf{B}$; the independent s-fold supplies that test. The cited one-parameter proximity gap and the t sampled positions reject any input farther than τ from the interleaved code, except with probability $3(k-1)n/|\mathbb{F}| + 3(1-\tau)^t$.

Otherwise, $\tau < \delta/2$ yields unique decoded matrices. If their product relation is false, structured Freivalds fails with probability at most $(k-1)/|\mathbb{F}|$. Conditioned on its success, at least one sampled encoded relation disagrees on a $(\delta-\tau)$-fraction of positions, contributing $4(1-\delta+\tau)^t$. Zero knowledge follows from the simulators of the CP-SNARK and CP-Link and the hiding of the Pedersen commitments. Appendix A gives the complete bad-event decomposition. □

Remark 1 (Fiat–Shamir). The implementation derives (r, s), I, and the code-check points from domain-separated transcript hashes. A formal analysis of this multi-round Fiat–Shamir transform is outside the interactive theorem and left to future work.

7. Implementation and Evaluation

We implement LAMP in Go using gnark over BN254. The CP-SNARK uses the Groth16 commit-and-prove interface [32], [33], and CP-Link is instantiated with the pairing-based QA-NIZK described in Appendix C. Our evaluation compares the revised protocol with a matched Freivalds-in-Groth16 baseline and with the public DualMatrix implementation. The implementation and experiment scripts will be released with the artifact.

Parameters and setup. Experiments run on a Linux server with an AMD EPYC 7B13 CPU, 32 cores, and 254 GB of memory. We use a rate-$1/2$ Reed–Solomon code and BN254. Following Section 6, the revised protocol uses $\tau = \delta/3$ and $t = 512$ queried coordinates. Groth16 has a circuit-specific setup, and the QALink CRS depends on the linked block layout and query count. We report setup separately from online proving and verification. For every reported point, we state the number of repetitions, the aggregation statistic, peak memory, and whether encoding and commitment generation are included.

7.1. Matched Freivalds Baseline

The Freivalds baseline computes the three vector–matrix products inside a Groth16 circuit. It is not a state-of-the-art matrix proof; using the same field, backend, hardware, and input generation isolates the effect of replacing those products with sampled encoded-column checks. We report constraints, setup time, encoding and commitment time, online proving and verification time, peak memory, and serialized proof size. A Groth16 proof remains constant-size as the circuit grows, whereas a LAMP proof also contains sampled Merkle openings and CP-Link material.

7.2. Comparison with Dedicated Matrix Proofs

Section 2 compares the asymptotic costs of zkMatrix, DualMatrix, zkMaP, and LAMP. We could not locate public zkMatrix code, and the zkMaP artifact link was unavailable, so a controlled direct measurement of those systems was not possible. DualMatrix is the publicly implemented follow-up to zkMatrix and is therefore the closest reproducible dedicated baseline. The comparison uses dense matrices, records the exact repository commit and build command, and reports both systems on the same machine. Because the proof statements and setup models differ, we report proving time, verification time, and proof size without presenting them as a component-by-component equivalence.

7.3. Component Costs and Application Workloads

The component breakdown separates input encoding and commitment, native intermediate-vector computation, intermediate encoding and commitment, Groth16 proving, Merkle openings, and CP-Link. This accounting is important because the added B test performs one additional native vector–matrix product and one additional encoding, while the full intermediate commitment changes the CP-Link block layout. Total prover work remains $\mathcal{O}(k^2)$, whereas the revised circuit remains $\mathcal{O}(tk + E_{\mathsf{code}})$.

Batching and the GPT-2 layer are secondary workloads. We include them only after rerunning them with the same revised transcript and soundness parameters. For GPT-2, the discussion reports prover-side gains together with verification and proof-size overhead; it does not describe a prover-only improvement as an end-to-end speedup.

8. Conclusion

We proposed LAMP, a protocol for verifying matrix multiplication by moving expensive vector–matrix products outside the SNARK circuit and checking them through sampled local tests over encoded data. The prover commits to the encoded matrices and Freivalds intermediate vectors, while the SNARK circuit proves only the sampled fold relations. Merkle openings and CP-Link proofs ensure that the values used inside the CP-SNARK are consistent with the externally committed data fixed before the sampling challenges are known.

For a fixed number t of queries, the main arithmetic relation has $\mathcal{O}(tk+E_{\mathsf{code}})$ constraints rather than the $\mathcal{O}(k^2)$ constraints of a Freivalds baseline. Thus, when t is fixed and the code-consistency checks are linear, the constraint count of LAMP grows linearly with the matrix dimension k. Total prover work remains quadratic because the prover computes native vector–matrix products and commits to the encoded inputs. Our evaluation therefore separates constraint savings from end-to-end proving, verification, and proof size, and compares the revised construction with both a matched Groth16 baseline and a public dedicated matrix-proof implementation.

The main remaining costs are sampled-opening proof size and pairing-based link verification. As future work, we plan to reduce opening and linking overhead, improve sharing across consecutive matrix products of different sizes, and exploit fixed model weights in verifiable AI scenarios for preprocessing and batching. These optimizations may reduce proof size and verification time on more complex AI workloads while preserving the linear constraint growth of LAMP.

References

[1] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*. Berlin, Heidelberg: Springer, 2016, pp. 305–326.

[2] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: https://eprint.iacr.org/2019/953

[3] M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn, "Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2019, pp. 2111–2128.

[4] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, "Marlin: Preprocessing zkSNARKs with universal and updatable SRS," in *Advances in Cryptology – EUROCRYPT 2020*. Cham: Springer, 2020, pp. 738–768.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[7] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[8] R. Freivalds, "Probabilistic machines can use less running time," in *Information Processing 77 (Proceedings of IFIP Congress 77)*. Amsterdam, The Netherlands: North-Holland, 1977, pp. 839–842.

[9] J. Thaler, "Time-optimal interactive proofs for circuit evaluation," in *Advances in Cryptology – CRYPTO 2013*. Berlin, Heidelberg: Springer, 2013, pp. 71–89.

[10] M. Cong, T. H. Yuen, and S.-M. Yiu, "zkMatrix: Batched short proof for committed matrix multiplication," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2024, pp. 289–305.

[11] M. Cong, T. H. Yuen, and S. M. Yiu, "DualMatrix: Conquering zkSNARK for large matrix multiplication," *Cybersecurity*, vol. 9, no. 1, p. 126, 2026.

[12] B. Deressa and M. A. Hasan, "zkmap: Zero-knowledge succinct non-interactive matrix multiplication proofs," *IACR Communications in Cryptology*, vol. 2, no. 3, 2025.

[13] S. Lee, H. Ko, J. Kim, and H. Oh, "vCNN: Verifiable convolutional neural network based on zk-SNARKs," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 4254–4270, 2024.

[14] J. Weng, J. Weng, G. Tang, A. Yang, M. Li, and J.-N. Liu, "pvcnn: Privacy-preserving and verifiable convolutional neural network testing," 2023. [Online]. Available: https://arxiv.org/abs/2201.09186

[15] Y. Zhang, M. Zheng, X. Chen, J. Hu, W. Shi, L. Ju, Y. Solihin, and Q. Lou, "zkVC: Fast zero-knowledge proof for private and verifiable computing," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, IEEE. New York, NY, USA: IEEE, 2025, pp. 1–7.

[16] T. Liu, X. Xie, and Y. Zhang, "zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2021, pp. 2968–2985.

[17] W. Qu, Y. Sun, X. Liu, T. Lu, Y. Guo, K. Chen, and J. Zhang, "zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference," Cryptology ePrint Archive, Paper 2025/1184, 2025. [Online]. Available: https://eprint.iacr.org/2025/1184

[18] H. Sun, J. Li, and H. Zhang, "zkllm: Zero knowledge proofs for large language models," 2024. [Online]. Available: https://arxiv.org/abs/2404.16109

[19] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, "Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning," in *30th USENIX Security Symposium (USENIX Security 21)*. Berkeley, CA, USA: USENIX Association, 2021, pp. 501–518.

[20] B.-J. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, "zkML: An optimizing system for ML inference in zero-knowledge proofs," in *Proceedings of the Nineteenth European Conference on Computer Systems*. New York, NY, USA: ACM, 2024, pp. 560–574.

[21] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast Reed-Solomon Interactive Oracle Proofs of Proximity," in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), I. Chatzigiannakis, C. Kaklamanis, D. Marx, and D. Sannella, Eds., vol. 107. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, pp. 14:1–14:17. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14

[22] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, "STIR: Reed–solomon proximity testing with fewer queries," Cryptology ePrint Archive, Paper 2024/390, 2024. [Online]. Available: https://eprint.iacr.org/2024/390

[23] ——, "WHIR: Reed–solomon proximity testing with super-fast verification," Cryptology ePrint Archive, Paper 2024/1586, 2024. [Online]. Available: https://eprint.iacr.org/2024/1586

[24] H. Zeilberger, B. Chen, and B. Fisch, "BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes," Cryptology ePrint Archive, Paper 2023/1705, 2023. [Online]. Available: https://eprint.iacr.org/2023/1705

[25] S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam, "Ligero: Lightweight sublinear arguments without a trusted setup," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2017, pp. 2087–2104.

[26] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, "Brakedown: Linear-time and field-agnostic snarks for r1cs," in *Advances in Cryptology – CRYPTO 2023*. Cham: Springer, 2023, pp. 193–226.

[27] M. Brehm, B. Chen, B. Fisch, N. Resch, R. D. Rothblum, and H. Zeilberger, "Blaze: Fast SNARKs from interleaved RAA codes," Cryptology ePrint Archive, Paper 2024/1609, 2024. [Online]. Available: https://eprint.iacr.org/2024/1609

[28] T. Xie, Y. Zhang, and D. Song, "Orion: Zero knowledge proof with linear prover time," Cryptology ePrint Archive, Paper 2022/1010, 2022. [Online]. Available: https://eprint.iacr.org/2022/1010

[29] T. den Hollander and D. Slamanig, "A crack in the firmament: Restoring soundness of the orion proof system and more," Cryptology ePrint Archive, Paper 2024/1164, 2024. [Online]. Available: https://eprint.iacr.org/2024/1164

[30] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf, "Proximity gaps for reed–solomon codes," *Journal of the ACM*, vol. 70, no. 5, pp. 31:1–31:57, 2023.

[31] B. E. Diamond and J. Posen, "Proximity testing with logarithmic randomness," *IACR Communications in Cryptology*, vol. 1, no. 1, 2024. [Online]. Available: https://doi.org/10.62056/aksdkp10

[32] A. Belling, A. Soleimanian, and O. Bégassat, "Recursion over public-coin interactive proof systems; faster hash verification," Cryptology ePrint Archive, Paper 2022/1072, 2022. [Online]. Available: https://eprint.iacr.org/2022/1072

[33] M. Campanelli, D. Fiore, and A. Querol, "LegoSNARK: Modular design and composition of succinct zero-knowledge proofs," Cryptology ePrint Archive, Paper 2019/142, 2019. [Online]. Available: https://eprint.iacr.org/2019/142

[34] J. T. Schwartz, "Fast probabilistic algorithms for verification of polynomial identities," *Journal of the ACM*, vol. 27, no. 4, pp. 701–717, 1980.

[35] R. Zippel, "Probabilistic algorithms for sparse polynomials," in *Symbolic and Algebraic Computation (EUROSAM '79)*. Berlin, Heidelberg: Springer, 1979, pp. 216–226.

Appendix A. Detailed Proof of Theorem 6.2

Proof. We prove the public-coin interactive protocol. The Fiat–Shamir transform used by the implementation is discussed separately at the end and is not covered by the theorem.

Perfect completeness. Assume that the honest prover holds matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$ with $\mathbf{AB} = \mathbf{C}$. Let $\mathbf{r} = (1, r, \ldots, r^{k-1})$ and $\mathbf{s} = (1, s, \ldots, s^{k-1})$, and define

$$\mathbf{x} = \mathbf{rA}, \qquad \mathbf{y} = \mathbf{xB}, \qquad \mathbf{z} = \mathbf{rC}, \qquad \mathbf{b} = \mathbf{sB}.$$

Then $\mathbf{y} = \mathbf{z}$. The honest prover commits to the packed encoded matrix blocks $\{\mathbf{m}_{ABC,j}\}_{j\in[n]}$ under rt_{ABC}, commits to the complete intermediate block $\mathbf{m}_{XYZB}$ under cm_{XYZB}, and supplies the sampled openings and CP-Link proofs required by Section 5.4.

For every queried position $j \in I$, code linearity gives

$$\mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*, j]) = \widehat{\mathbf{x}}[j],$$

and similarly

$$\mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*, j]) = \widehat{\mathbf{y}}[j],$$
$$\mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*, j]) = \widehat{\mathbf{z}}[j],$$
$$\mathsf{Fold}(\mathbf{s}, \widehat{\mathbf{B}}[*, j]) = \widehat{\mathbf{b}}[j].$$

Since $\mathbf{y} = \mathbf{z}$, the equality check $\widehat{\mathbf{y}}[j] = \widehat{\mathbf{z}}[j]$ also holds. All SNARK, Merkle, and CP-Link checks are honestly generated, so the verifier accepts.

Bad-event decomposition. Let Acc be the event that the verifier accepts a false committed matrix multiplication statement. We decompose the failure probability into the following events:

- $\mathsf{Bad}_{\mathsf{SNARK}}$: the CP-SNARK accepts although the sampled relation of Section 5.3 is false for the committed sampled witnesses;
- $\mathsf{Bad}_{\mathsf{open}}$: a Merkle opening or Pedersen leaf opening is accepted in a way that violates position binding or commitment binding;
- $\mathsf{Bad}_{\mathsf{link}}$: a CP-Link proof accepts although the SNARK-side sampled ABC blocks are not the corresponding blocks opened under rt_{ABC}, or the complete SNARK-side intermediate block is not the message committed by cm_{XYZB};
- $\mathsf{Bad}_{\mathsf{code}}$: the probabilistic code-consistency checks accept encoded views that are not encodings of their corresponding intermediate vectors;

- $\mathsf{Bad}_{\mathsf{uni}}$: an input word farther than τ from the interleaved code produces a structured fold within distance τ of $\mathcal{C}$.

By the assumed soundness of the selected backends,

$$\begin{aligned}\Pr[\mathsf{Bad}_{\mathsf{SNARK}}] &\le \epsilon_{\mathsf{cp}}(\lambda),\\ \Pr[\mathsf{Bad}_{\mathsf{open}}] &\le \epsilon_{\mathsf{open}}(\lambda),\\ \Pr[\mathsf{Bad}_{\mathsf{link}}] &\le \epsilon_{\mathsf{link}}(\lambda),\\ \Pr[\mathsf{Bad}_{\mathsf{code}}] &\le \epsilon_{\mathsf{code}}(\lambda),\\ \Pr[\mathsf{Bad}_{\mathsf{uni}}] &\le \frac{3(k-1)n}{|\mathbb{F}|}.\end{aligned}$$

Condition on the complement of these events. Then rt_{ABC} fixes a unique sequence of packed matrix-column commitments at all queried positions, cm_{XYZB} fixes the complete messages and encoded views used by the SNARK, and the sampled input values are exactly the messages opened under rt_{ABC}. In particular, the adversary cannot choose a different intermediate encoding after seeing I or the code-check points without breaking the linking layer.

Input proximity and extraction. The root rt_{ABC} fixes arbitrary words $W_A, W_B, W_C \in \mathbb{F}^{k\times n}$ before (r, s) is sampled. Suppose first that one of them is farther than τ from the interleaved code. We use r for W_A, W_C and the independent s for W_B. By the one-parameter Reed–Solomon proximity gap of Ben-Sasson et al. [30, Theorem 1.5], conditioned on $\neg\mathsf{Bad}_{\mathsf{uni}}$, the corresponding folded word is farther than τ from $\mathcal{C}$. The encoded view claimed by the prover is a codeword conditioned on $\neg\mathsf{Bad}_{\mathsf{code}}$, so the two words disagree on more than τn positions. Since both are committed before I, the probability that t independent queries miss this set is at most $(1-\tau)^t$. Union-bounding over A, B, C gives $3(1-\tau)^t$.

It remains to consider the case in which all three words are within distance τ of the interleaved code. Because $\tau < \delta/2$, each is close to a unique interleaved codeword. Decode those codewords row-wise to obtain fixed matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$. These are the matrices bound by the accepted committed statement.

Freivalds randomization. Suppose $\mathbf{AB} \ne \mathbf{C}$, and let $\mathbf{D} = \mathbf{AB} - \mathbf{C} \ne 0$. Some column $\mathbf{d}$ of $\mathbf{D}$ is nonzero. The event $\mathbf{rD} = 0$ implies

$$P(r) := \langle (1, r, \ldots, r^{k-1}), \mathbf{d}\rangle = \sum_{i=0}^{k-1} d_i r^i = 0.$$

The polynomial P is nonzero and has degree at most $k-1$. Since r is sampled after rt_{ABC} is fixed, Schwartz–Zippel gives

$$\Pr[\mathbf{rD} = 0] \le \frac{k-1}{|\mathbb{F}|}.$$

Sampled relation checks. Now also condition on $\mathbf{rD} \ne 0$. No vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ can satisfy all four global relations

$$\mathbf{x} = \mathbf{rA}, \qquad \mathbf{y} = \mathbf{xB}, \qquad \mathbf{z} = \mathbf{rC}, \qquad \mathbf{y} = \mathbf{z}.$$

Otherwise they would imply $\mathbf{rAB} = \mathbf{rC}$, contradicting $\mathbf{rD} \ne 0$.

Therefore at least one checked encoded relation is false. Consider the case $\mathbf{x} \ne \mathbf{rA}$; the other three cases are identical. The codewords $\mathsf{Enc}(\mathbf{x})$ and $\mathsf{Enc}(\mathbf{rA})$ are distinct, so by relative distance they disagree on at least δn coordinates. The committed word W_A differs from $\mathsf{Enc}(\mathbf{A})$ at no more than τn column positions. Therefore $\mathsf{FoldWord}(\mathbf{r}, W_A)$ and $\mathsf{Enc}(\mathbf{x})$ disagree at no fewer than $(\delta - \tau)n$ positions. For any queried coordinate j, code linearity gives

$$\mathsf{Enc}(\mathbf{rA})[j] = \mathsf{Fold}(\mathbf{r}, \mathsf{Enc}(\mathbf{A})[*, j]).$$

The sampled-opening backend ensures that the SNARK uses the opened $W_A[*, j]$ block under rt_{ABC}, while the full intermediate link ensures that $\widehat{\mathbf{x}}[j]$ is the symbol fixed by cm_{XYZB}. Thus the fold check rejects whenever j lands in the disagreement set. A uniformly sampled coordinate misses this set with probability at most $1 - \delta + \tau$, and t independent samples miss it with probability at most $(1-\delta+\tau)^t$. Union-bounding over the four possible violated relations gives

$$4(1 - \delta + \tau)^t.$$

Combining the bounds. Adding the SNARK, opening, linking, code-consistency, Freivalds, and proximity failure events gives

$$\begin{aligned}\Pr[\mathsf{Acc} \wedge \mathsf{BadInput}] \le\ & \epsilon_{\mathsf{cp}}(\lambda) + \epsilon_{\mathsf{open}}(\lambda) + \epsilon_{\mathsf{link}}(\lambda)\\ &+ \epsilon_{\mathsf{code}}(\lambda) + \frac{3(k-1)n}{|\mathbb{F}|}\\ &+ 3(1-\tau)^t + \frac{k-1}{|\mathbb{F}|}\\ &+ 4(1-\delta+\tau)^t.\end{aligned}$$

For the rate-1/2 Reed–Solomon code, setting $\tau = \delta/3 \approx 1/6$ and $t = 512$ makes the sampling contribution $3(1-\tau)^t + 4(1-\delta+\tau)^t$ smaller than 2^{-133}. For BN254 and the evaluated dimensions, the two field-size terms and the encoding-check term are smaller still. The final security level also includes the computational security of the CP-SNARK, commitments, and CP-Link.

Zero knowledge. Consider a hybrid that replaces the CP-SNARK proof by its simulator output and then replaces the CP-Link proofs by their simulator outputs. The hiding of the Pedersen commitments conceals both the sampled input columns and the complete intermediate block; the Merkle paths reveal only commitments at the sampled positions. The resulting view is simulatable from the public statement, and the hybrid transitions are indistinguishable by the assumed zero-knowledge and hiding properties. Together with perfect completeness and the soundness bound above, this proves Theorem 6.2 for the public-coin interactive protocol.

Fiat–Shamir variant. The implementation derives (r, s), I, and backend batch challenges from domain-separated hashes of the transcript. A formal random-oracle analysis of this multi-round Fiat–Shamir transform is outside Theorem 6.2 and is left to future work. □

Appendix B. Barycentric Reed–Solomon Consistency Check

This appendix specifies the Reed–Solomon instantiation of $\mathsf{EncCheck}_{\mathcal{C}}$ used in Section 5.3. Let

$$D = \{\omega_0, \dots, \omega_{n-1}\} \subset \mathbb{F}$$

be the public evaluation domain. A message $\mathbf{v} = (v_0, \dots, v_{k-1}) \in \mathbb{F}^k$ defines the degree-$< k$ polynomial

$$p_{\mathbf{v}}(X) = \sum_{\ell=0}^{k-1} v_\ell X^\ell, \qquad \mathsf{Enc}(\mathbf{v}) = (p_{\mathbf{v}}(\omega_i))_{i=0}^{n-1}.$$

Given a claimed encoded word $\widehat{\mathbf{v}} \in \mathbb{F}^n$, let $P_{\widehat{\mathbf{v}}}$ be the unique degree-$< n$ polynomial satisfying

$$P_{\widehat{\mathbf{v}}}(\omega_i) = \widehat{\mathbf{v}}[i] \qquad \text{for all } i \in [n].$$

The check samples $\alpha \leftarrow\!\!\$\ \mathbb{F} \setminus D$ after the claimed word is fixed and compares $P_{\widehat{\mathbf{v}}}(\alpha)$ with $p_{\mathbf{v}}(\alpha)$.

For efficient evaluation, define the domain polynomial and barycentric weights

$$L_D(X) = \prod_{i=0}^{n-1}(X - \omega_i), \quad \lambda_i = \left(\prod_{\ell \neq i}(\omega_i - \omega_\ell)\right)^{-1}.$$

For every $\alpha \notin D$, barycentric interpolation gives

$$P_{\widehat{\mathbf{v}}}(\alpha) = L_D(\alpha) \sum_{i=0}^{n-1} \lambda_i \frac{\widehat{\mathbf{v}}[i]}{\alpha - \omega_i}.$$

Thus the concrete Reed–Solomon consistency check is

$$\mathsf{EncCheck}_{\mathsf{RS}}(\widehat{\mathbf{v}}, \mathbf{v}; \alpha) = 1 \iff \sum_{\ell=0}^{k-1} v_\ell \alpha^\ell = L_D(\alpha) \sum_{i=0}^{n-1} \lambda_i \frac{\widehat{\mathbf{v}}[i]}{\alpha - \omega_i}.$$

The right-hand side is a linear form in the claimed encoded symbols once α is fixed. Equivalently, the verifier can derive the interpolation coefficients

$$\gamma_i(\alpha) = L_D(\alpha)\lambda_i(\alpha - \omega_i)^{-1}$$

and the circuit checks

$$\sum_{\ell=0}^{k-1} v_\ell \alpha^\ell = \sum_{i=0}^{n-1} \gamma_i(\alpha)\widehat{\mathbf{v}}[i].$$

This costs $\mathcal{O}(k+n)$ field operations per checked point, after the domain-dependent weights are precomputed.

Lemma B.1 (Barycentric code-consistency soundness)**.** *If $\widehat{\mathbf{v}} \neq \mathsf{Enc}(\mathbf{v})$ and $\alpha \leftarrow\!\!\$\ \mathbb{F} \setminus D$ is sampled after $\widehat{\mathbf{v}}$ and $\mathbf{v}$ are fixed, then*

$$\Pr[\mathsf{EncCheck}_{\mathsf{RS}}(\widehat{\mathbf{v}}, \mathbf{v}; \alpha) = 1] \leq \frac{n-1}{|\mathbb{F}| - n}.$$

Proof. If $\widehat{\mathbf{v}} \neq \mathsf{Enc}(\mathbf{v})$, then the interpolation polynomial $P_{\widehat{\mathbf{v}}}$ differs from the degree-$< k$ polynomial $p_{\mathbf{v}}$. Their difference

$$Q(X) = P_{\widehat{\mathbf{v}}}(X) - p_{\mathbf{v}}(X)$$

is a nonzero polynomial of degree at most $n-1$. The check accepts exactly when $Q(\alpha) = 0$. Since α is uniform over $\mathbb{F} \setminus D$, at most $n-1$ choices of α can make the check accept, which proves the bound. □

In LAMP, this check is applied independently to $\widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}$ at $\alpha_x, \alpha_y, \alpha_z, \alpha_b$. A union bound gives the $4(n-1)/(|\mathbb{F}| - n)$ term used in Definition 6.1.

Appendix C. CP-Link Instantiation

Our implementation uses a pairing-based quasi-adaptive link argument (QALink). For q blocks $\mathbf{m}_i \in \mathbb{F}^\ell$, the public statement contains one SNARK-side Pedersen commitment $\widetilde{\mathsf{cm}}$ and external commitments $\mathsf{cm}_0, \dots, \mathsf{cm}_{q-1}$ satisfying

$$\widetilde{\mathsf{cm}} = \sum_{i \in [q]} \sum_{\nu \in [\ell]} \mathbf{m}_i[\nu] G^S_{i,\nu} + \widetilde{o} H^S,$$

$$\mathsf{cm}_i = \sum_{\nu \in [\ell]} \mathbf{m}_i[\nu] G^E_\nu + o_i H^E.$$

Setup samples nonzero $k_0, \dots, k_q, a \in \mathbb{F}$, publishes the proving-key elements $P_{i,\nu} = k_0 G^S_{i,\nu} + k_{i+1} G^E_\nu$, and publishes $C_i = a k_i g_2$ and $A = a g_2$ in the verification key. The prover outputs

$$\pi_{\mathsf{link}} = k_0 \widetilde{\mathsf{cm}} + \sum_{i \in [q]} k_{i+1} \mathsf{cm}_i,$$

expressed using the proving key and the openings. The verifier checks

$$e(\widetilde{\mathsf{cm}}, C_0) \prod_{i \in [q]} e(\mathsf{cm}_i, C_{i+1}) = e(\pi_{\mathsf{link}}, A).$$

Soundness follows from the quasi-adaptive linear-subspace assumption for this CRS. One proof contains one $\mathbb{G}_1$ element; verification uses $q+2$ pairing terms, which a multi-pairing implementation evaluates with one final exponentiation. The proving and verification keys depend on q and ℓ, so changing the sample count or linked block layout requires new setup material.

Multiple link equations are batched by deriving a nonzero transcript coefficient λ_h for each equation $E_h = 1$ and checking $\prod_h E_h^{\lambda_h} = 1$ in one multi-pairing. If at least one equation is invalid, a fresh coefficient vector satisfies the resulting nontrivial linear equation with probability at most $1/(|\mathbb{F}|-1)$. The non-interactive implementation derives these coefficients with Fiat–Shamir.